TRABALHO 2 • SISTEMAS DISTRIBUÍDOS

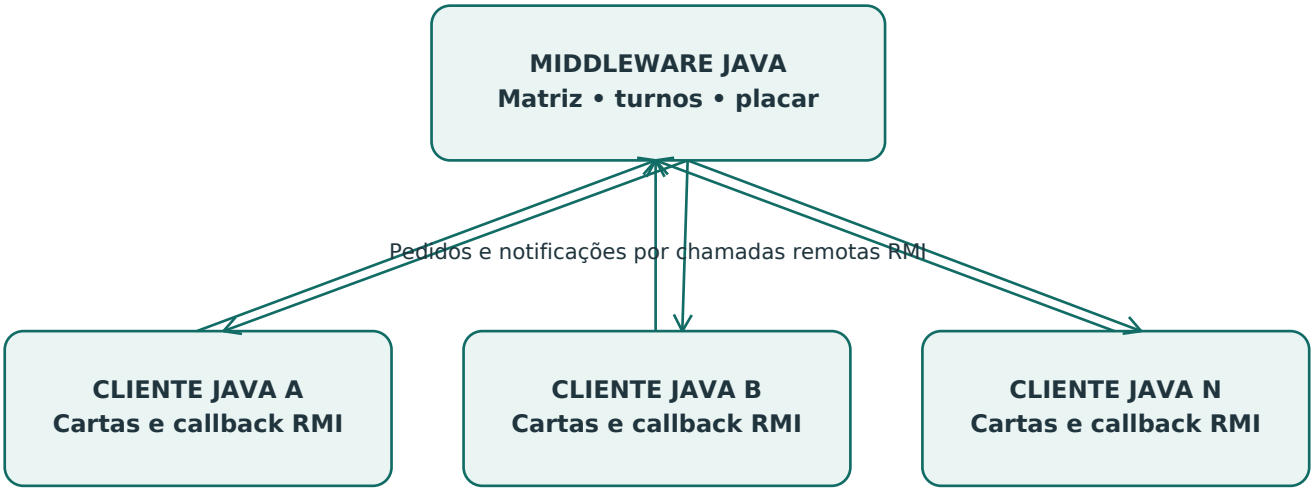
Jogo da memória online com Java RMI

Desenvolva um jogo da memória distribuído, disputado online por diversos jogadores. Todos os módulos deverão ser implementados em Java, e a comunicação entre os clientes e o middleware deverá utilizar, obrigatoriamente, Java RMI.

Objetivo e entrega

Este trabalho deve demonstrar compreensão de invocação remota, coordenação centralizada, concorrência e consistência de estado entre processos. O aluno deverá entregar um relatório individual em um arquivo PDF próprio, separado do primeiro trabalho, explicando a implementação e apresentando evidências de funcionamento. O professor avaliará os conhecimentos demonstrados e a pontuação correspondente.

1. Arquitetura obrigatória



Componente	Responsabilidade
Middleware Java	Mantém o estado oficial, sorteia as cartas, valida jogadas, controla turnos e calcula os resultados.
Clientes Java	Interfaces gráficas para jogadores, com tabuleiro, imagens, placar e indicação de quem deve jogar.
Infraestrutura RMI	Registro e localização do serviço remoto; interfaces remotas e objetos de callback para notificações aos clientes.

A sala deve suportar pelo menos dois jogadores e permitir a participação de mais de dois. A capacidade máxima pode ser definida pelo aluno e documentada. Não basta executar um jogo local com interfaces que simulam jogadores: cada cliente deve ser um processo independente conectado ao middleware.

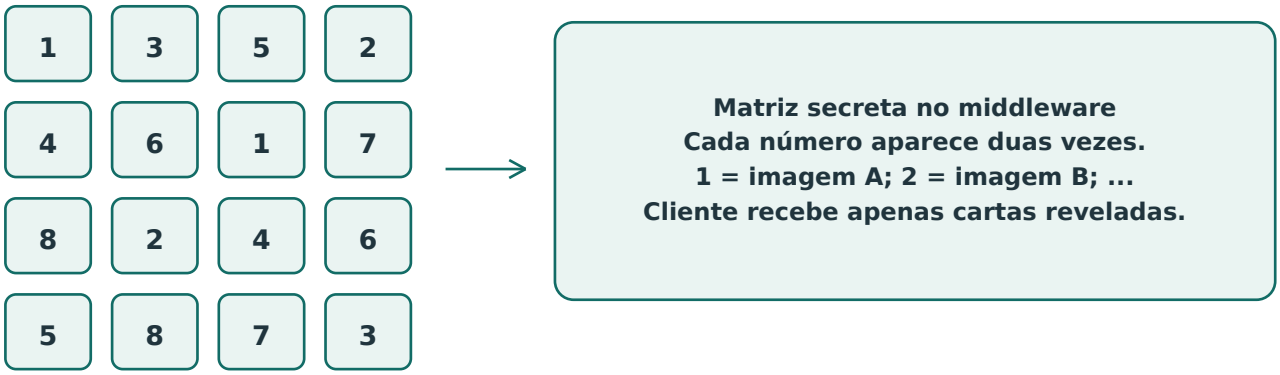
Tabuleiro, imagens e regras da partida

2. Matriz e sorteio no middleware

• O middleware deve criar uma matriz de cartas com quantidade par de posições. Cada identificador numérico deve ocorrer exatamente duas vezes, formando um par associado à mesma imagem.

• Gere os identificadores de pares e embaralhe suas posições aleatoriamente antes de cada partida. Evite números sorteados sem controle, que possam gerar cartas sem par ou mais de duas ocorrências.

• As dimensões e o conjunto de imagens ficam a critério do aluno. Use um tabuleiro com vários pares; o exemplo abaixo é ilustrativo. Os clientes devem iniciar com todas as cartas fechadas.



Exemplo ilustrativo: tabuleiro 4 × 4, com 8 pares.

3. Execução da jogada

• O middleware define a ordem dos jogadores e informa a todos qual jogador possui a vez. A regra de escolha do primeiro jogador deve ser documentada.

• Uma jogada válida consiste em selecionar duas posições distintas ainda disponíveis. A cada clique aceito, o middleware revela a carta e notifica todos os clientes para exibir a imagem correspondente.

• Se as duas cartas formarem um par, elas permanecem abertas e deixam de estar disponíveis. O jogador ganha um par no placar e continua com a vez enquanto acertar novos pares.

• Se as cartas forem diferentes, todos devem poder vê-las por um breve intervalo documentado. Em seguida, elas são fechadas e a vez passa ao próximo jogador, em ordem circular.

• Durante a exibição de um par incorreto, não aceite novos cliques. O middleware coordena a resolução e a troca de turno; os clientes não decidem sozinhos o resultado.

Regra de encerramento

A partida termina quando todos os pares forem encontrados. Vence quem tiver obtido mais pares nessa partida. Se houver empate no maior número de pares, anuncie empate entre os jogadores correspondentes. O middleware deve impedir novas jogadas e enviar o resultado a todos via RMI.

Chamadas remotas e estado consistente

4. Contrato de comunicação por RMI

• Defina interfaces remotas para entrada na sala, preparação da partida, seleção de cartas, consulta do estado e resposta sobre continuar ou encerrar a sessão. Os métodos remotos devem declarar o tratamento de falhas de comunicação.

• Cada cliente deve disponibilizar um objeto remoto de callback, registrado no middleware, para receber atualizações do tabuleiro, mudança de turno, placar, resultado da partida e sumário final.

• O aviso a todos os participantes deve ser feito por chamadas RMI aos callbacks registrados. Não utilize multicast de rede como substituto dessas notificações. O aviso deve alcançar todos os jogadores da sala, mesmo quando houver mais de dois.

• Documente interfaces, métodos, parâmetros e estruturas de dados trocadas. Objetos transportados por valor devem ser serializáveis quando necessário; objetos remotos devem ser exportados adequadamente.

Fluxo de uma seleção de carta

Etapa	Ação
1. Cliente	Envia via RMI a identidade da sessão/participante, a partida e a posição selecionada.
2. Middleware	Valida jogador, turno, partida, posição e disponibilidade da carta.
3. Estado oficial	Atualiza a carta revelada; após a segunda seleção, resolve acerto ou erro e atualiza o placar/turno.
4. Notificação RMI	Comunica o estado público aos clientes por callback.
5. Clientes	Atualizam tabuleiro, imagens, placar e indicação de vez de forma coerente.

5. Concorrência e validação

• Rejeite jogadas fora da vez, seleção repetida da mesma carta, posições inválidas, cartas já conquistadas e comandos referentes a uma partida encerrada ou anterior.

• O servidor é a autoridade sobre tabuleiro, turnos e pontuação. Desabilitar botões no cliente não substitui a validação no middleware.

• Garanta que chamadas simultâneas não produzam duas jogadas conflitantes nem contagem duplicada de pares. Explique a estratégia para proteger o estado compartilhado e ordenar as atualizações.

• Não envie ao cliente a matriz secreta completa antes das revelações. Pode distribuir previamente um catálogo de imagens, sem divulgar a associação das posições ocultas às imagens.

• Trate falhas RMI e desconexões com uma política documentada, por exemplo pausar ou cancelar a partida afetada. Um cliente indisponível não deve bloquear indefinidamente os demais. Explique como callbacks e interface gráfica são coordenados.

Reinício, estatísticas e campeão

6. Partida e sessão

Uma partida corresponde a um tabuleiro até a descoberta de todos os pares. Uma sessão de jogo reúne uma ou mais partidas do mesmo grupo e termina quando os participantes não desejarem continuar. As estatísticas acumuladas pertencem à sessão.

• Ao finalizar uma partida, o middleware envia via RMI o vencedor ou empate, os pares de cada jogador e a opção de jogar novamente.

• Colete a resposta de cada participante. Para este enunciado, uma nova partida do mesmo grupo inicia somente se todos concordarem; qualquer resposta negativa encerra a sessão e gera o sumário para todos. Ausência de resposta deve seguir um prazo/política documentado.

• No reinício, sorteie uma nova matriz, feche as cartas e zere o placar da partida. Preserve o número de partidas concluídas e as estatísticas acumuladas da sessão.

7. Cálculo do aproveitamento

Considere uma tentativa como a seleção válida e completa de duas cartas. Um acerto equivale a um par encontrado. Cliques isolados, comandos rejeitados e seleções incompletas não contam como tentativas. O cálculo deve ser feito pelo middleware:

Indicador	Definição
Aproveitamento (%)	100 × pares encontrados ÷ tentativas completas
Nenhuma tentativa	Exibir 0% e informar que o jogador ainda não realizou tentativas.
Acumulado da sessão	100 × soma dos pares ÷ soma das tentativas; não usar a média simples dos percentuais das partidas.

Exemplo ilustrativo de uma sessão com 3 partidas concluídas

Jogador	Vitórias	Pares	Tentativas	Aproveitamento
Ana	2	14	20	70%
Bruno	1	10	25	40%

8. Sumário e campeão da sessão

• Envie a todos, via RMI, o total de partidas concluídas e, por jogador: vitórias, pares encontrados, tentativas e percentual de aproveitamento. Mostre os resultados na interface de cada cliente.

• Para tornar a avaliação objetiva, adote: campeão é quem tiver mais vitórias; em empate, compare o total de pares; persistindo o empate, compare o aproveitamento acumulado. Se os três critérios empatarem, declare campeões empatados.

• Partidas empatadas devem ser identificadas no sumário e não atribuem vitória individual. Partidas canceladas por falhas não contam como concluídas; documente seu registro separadamente e não as misture às estatísticas finais.

As regras de continuação e de desempate acima explicitam decisões necessárias ao enunciado. O aluno deve aplicá-las de forma uniforme no middleware e explicar seus resultados no relatório.

Entrega, demonstração e avaliação

9. Conteúdo do PDF entregue pelo aluno

• Identificação; tecnologias e versão do Java; arquitetura e implantação com processos, máquinas, portas, RMI Registry e callbacks.

• Explicação do sorteio, matriz, associação a imagens, turnos, validações e controle de concorrência; contratos RMI e trechos de código relevantes.

• Capturas de tela e evidências de partidas com dois e com mais de dois clientes, incluindo acerto, erro, troca de turno, vitória, empate e reinício.

• Evidências do sumário, cálculos de aproveitamento e regra do campeão; tratamento de desconexões e limitações conhecidas.

• Instruções para compilar e executar middleware e clientes, com configuração de rede e localização do serviço. Inclua link acessível ao código-fonte e aos recursos de imagens para permitir verificação da implementação.

10. Roteiro mínimo de demonstração

1. Iniciar middleware e conectar jogadores em processos independentes.
2. Mostrar o tabuleiro fechado e a indicação da vez em todos os clientes.
3. Revelar duas cartas iguais: confirmar o par e a permanência da vez.
4. Revelar cartas diferentes: confirmar fechamento e passagem da vez.
5. Tentar jogada fora da vez e selecionar posição inválida ou indisponível.
6. Demonstrar atualizações sincronizadas com pelo menos três jogadores.
7. Encerrar uma partida e conferir a notificação RMI do resultado.
8. Reiniciar com concordância de todos e verificar novo sorteio.
9. Encerrar a sessão por resposta negativa e conferir sumário e campeão.
10. Simular empate, chamadas concorrentes e desconexão controlada.

11. Critérios de avaliação

O professor avaliará o atendimento aos requisitos, o funcionamento demonstrado e a capacidade de explicar a solução. Distribuição sugerida em percentuais da nota de cada trabalho (total de 100%), ajustável pelo professor:

Critério	Percentual da nota
Arquitetura Java, RMI, registro e callbacks	20%
Matriz de pares, sorteio e imagens	10%
Validação de jogadas, regras e controle dos turnos	20%
Interfaces gráficas e sincronização entre clientes	15%
Concorrência e tratamento de falhas remotas	10%
Fim de partida, reinício, sumário e campeão	15%
Relatório, evidências e execução reproduzível	10%

Java em todos os módulos e comunicação por RMI são requisitos obrigatórios. Uma implementação sem esses recursos não atende à proposta. Prazo e valor na disciplina serão informados pelo professor.